Hierarchical Edge-Cloud Federated Learning Framework for Scalable Time-Series Anomaly Detection Using Transformer-Based Models

Shailesh Mishra
Dept. of Electrical Engineering
Indian Institute of Technology Delhi
New Delhi, India
shaileshmishra601@gmail.com

Sougata Mukherjea
Dept. of Electrical Engineering
Indian Institute of Technology Delhi
New Delhi, India
sougatam@ee.iitd.ac.in

***Abstract*—Federated Learning (FL) has emerged as a privacy-preserving distributed machine learning paradigm, yet conventional flat client-server architectures suffer from communication bottlenecks and scalability limitations in large-scale IoT edge-cloud environments. Non-IID data distributions further degrade convergence, with FedAvg accuracy dropping by up to 55% under highly skewed partitions. Existing federated anomaly detection systems rely on CNNs, autoencoders, and LSTMs that struggle to model long-range temporal dependencies. This paper proposes a hierarchical edge-cloud federated learning framework for scalable distributed time-series anomaly detection using the Anomaly Transformer. The framework incorporates a Federated Transfer Learning (FTL) initialization stage, two-tier hierarchical edge aggregation, and comparative evaluation of FedAvg, FedProx, and FedAdaProx optimization strategies under non-IID conditions. The system is deployed on Microsoft Azure with Docker-based distributed clients and evaluated on UCI-HAR and N-BaIoT benchmark datasets. FedProx achieves 95.61% accuracy and 95.41% F_1-score on UCI-HAR, while FedAdaProx achieves 98.58% accuracy and 98.52% F_1-score on N-BaIoT, validating the effectiveness of the proposed framework for practical distributed anomaly detection in edge-cloud IoT environments.**

***Index Terms*—Federated Learning, Hierarchical FL, Federated Transfer Learning, Anomaly Transformer, Edge Computing, Anomaly Detection**

Published in: IEEE CloudCom 2026; 17th International Conference on Cloud Computing Technology and Science; Paris, France

I. Introduction

The rapid proliferation of Internet of Things (IoT) devices and edge computing platforms has produced massive volumes of time-series data requiring scalable, privacy-aware anomaly detection. Applications spanning smart cities, industrial automation, healthcare monitoring, and cybersecurity continuously generate distributed data streams where detecting abnormal patterns—cyberattacks, equipment failures, or unusual operational behaviors—is critical for system reliability [22]. Traditional centralized machine learning exposes raw data during transmission, introducing privacy risks, high communication overhead, and scalability bottlenecks as device populations grow [2].

Federated Learning (FL) [1] enables collaborative training without raw data sharing: clients train locally and transmit only model parameters to an aggregation server. However, non-IID data distributions across clients degrade convergence significantly—Zhao et al. [21] show FedAvg accuracy can drop by up to 55% under highly skewed partitions due to weight divergence proportional to the earth-mover distance between client and population distributions.

Moreover, flat FL architectures become communication-bottlenecked as client populations grow [20]. Hierarchical Federated Learning (HFL) [6] mitigates this via intermediate edge aggregation nodes. Since sharing parameters alone leaves inference risks, formal mechanisms like secure aggregation [32] and differential privacy [33] remain necessary.

Existing federated anomaly detection systems rely on LSTM encoder-decoders [29], CNNs, and autoencoders [8], [9] that capture only short-range temporal context. Transformer-based detectors [12], [25] model long-range dependencies through self-attention and consistently outperform recurrent baselines on multivariate time-series benchmarks [28]. Federated Transfer Learning (FTL) [3] further addresses non-IID convergence instability by initializing all clients from a common pretrained model, providing shared representations that reduce inter-client divergence.

This paper proposes a hierarchical edge-cloud FL framework integrating FTL, the Anomaly Transformer, and adaptive federated optimization on Microsoft Azure. Our contributions are:

- A practical three-tier hierarchical edge-cloud FL architecture deployed on Azure VMs with Docker clients, validating real-world feasibility beyond simulation.
- Integration of the Anomaly Transformer within hierarchical FL for distributed time-series anomaly detection via association discrepancy learning.
- An FTL initialization strategy distributing pretrained weights through the complete hierarchy before federated optimization.
- FedAdaProx: an adaptive proximal algorithm following the adaptive federated optimization principle [18] with

dynamic per-client regularization based on divergence and loss instability.
- Comparative evaluation of FedAvg, FedProx, and FedAdaProx on UCI-HAR and N-BaIoT under non-IID partitioning.

II. Related Work

A. Federated Learning and Optimization

McMahan et al. [1] introduced FedAvg for communication-efficient collaborative training. Karimireddy et al. [19] formally identified *client drift*—the divergence of local updates from the global gradient direction under non-IID data—as the root cause of FedAvg's convergence slowdown, and proposed SCAFFOLD's control variates (variance reduction) to correct it. Two complementary families address drift: proximal restraint (FedProx [13]) anchors local updates to the global model, while variance reduction (SCAFFOLD [19]) corrects the gradient direction directly. Reddi et al. [18] showed that applying adaptive server-side optimizers (Adam, Yogi) to the aggregated pseudo-gradient—yielding FedAdam, FedYogi—improves convergence under heterogeneity; they further proved the FedAvg default of server learning rate $= 1$ is suboptimal. Zhao et al. [21] quantified that FedAvg accuracy drops up to 55% on highly skewed partitions, motivating FedProx and FedAdaProx. Konecny et al. [20] established communication as the dominant FL bottleneck and proposed structured/sketched updates to reduce uplink cost.

B. Hierarchical Federated Learning

Liu et al. [6] proposed HierFAVG, the original three-tier client-edge-cloud FL scheme with a convergence analysis and guidance on edge-vs-cloud aggregation frequency—the direct ancestor of our architecture. Lim et al. [7] surveyed FL in mobile edge networks. Zhou et al. [22] provided the authoritative survey on edge intelligence, supporting the latency and privacy benefits of intermediate aggregation. Wu et al. [23] proposed HiFlash with adaptive staleness control and heterogeneity-aware client-edge association for asynchronous HFL; unlike HiFlash, our framework uses synchronous rounds to keep deployment reproducible. Sarhan et al. [24] proposed HBFL, a hierarchical blockchain-secured federated Intrusion Detection System (IDS) for IoT; HBFL uses no transformer-based detection and no real cloud deployment, precisely the axes our framework advances.

C. Federated Transfer Learning

Yang et al. [3] introduced FTL demonstrating that pretrained initializations improve convergence in heterogeneous non-IID settings. Limited research integrates FTL within hierarchical FL frameworks for transformer-based time-series detection, motivating its incorporation here.

D. Anomaly Detection

Darban et al. [28] taxonomized deep time-series anomaly detectors by strategy (reconstruction, prediction, association) and model family. Malhotra et al. [29] introduced LSTM encoder-decoder reconstruction for multi-sensor anomaly detection; Pang et al. [8] and Zhou and Paffenroth [9] extended these to robust autoencoders. Deng and Hooi [30] proposed GDN, learning inter-sensor dependency graphs for anomaly detection—an orthogonal direction to attention-based temporal modeling. Xu et al. [12] introduced the Anomaly Transformer with association discrepancy learning: normal points exhibit Gaussian-concentrated temporal associations, while anomalous points display disrupted patterns measurable by KL divergence. Tuli et al. [25] proposed TranAD, further confirming transformer advantages for multivariate anomaly detection with up to 17% F_1 gains. Wu et al. [27] proposed Autoformer with auto-correlation for long-horizon forecasting, and Zhou et al. [26] proposed Informer with ProbSparse attention—both provide efficient attention backbones relevant to resource-constrained edge deployment.

E. Federated IoT Anomaly Detection

Nguyen et al. [31] proposed DIoT, the first federated self-learning anomaly IDS for IoT targeting Mirai-type botnets—the closest prior work to our N-BaIoT evaluation. DIoT uses flat FL with no edge tier, non-transformer models, and no real cloud deployment, exactly the three limitations our framework advances. Bonawitz et al. [32] proposed practical secure aggregation preventing the server from observing individual client updates. Abadi et al. [33] introduced DP-SGD providing formal (ϵ, δ)-differential privacy guarantees for deep learning.

F. Research Gap

No prior work simultaneously addresses: transformer-based time-series anomaly detection in hierarchical FL, FTL initialization, real Azure cloud deployment with Docker orchestration, and adaptive proximal optimization under non-IID conditions.

III. System Architecture

A. Overall Framework

The proposed system adopts a three-tier hierarchical FL architecture (Fig. 1) following the client-edge-cloud paradigm of HierFAVG [6], extended with transformer-based detection and real Azure deployment. The architecture comprises:

1) **FTL Initialization Layer**: Pretrains the Anomaly Transformer and distributes pretrained weights through the hierarchy to all clients before round 1.
2) **Global Aggregation Layer**: One Azure VM coordinating federated rounds and performing global weighted aggregation.
3) **Edge Aggregation Layer**: Two Azure VMs performing intermediate regional aggregation, each managing three Docker clients.
4) **Distributed Client Layer**: Six Docker containers independently training on private data partitions without sharing raw data.

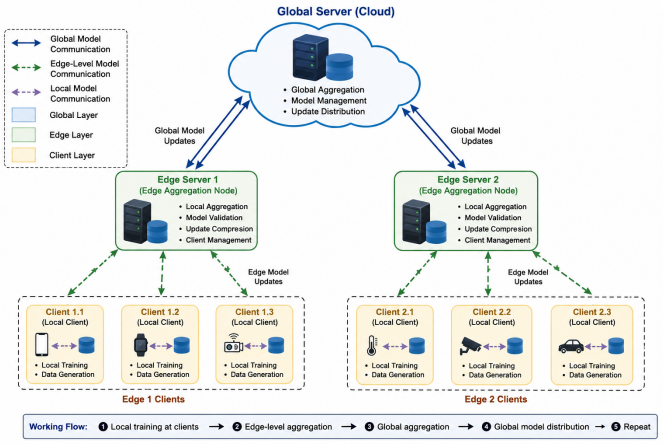


Fig. 1. Proposed Hierarchical Edge-Cloud FL Framework integrating FTL initialization, two-tier hierarchical aggregation, and six Docker-based clients on Microsoft Azure.

B. Aggregation Equations

Global aggregation combines edge-level models weighted by dataset size:

$$W_g^t = \sum_{e=1}^{m} \frac{N_e}{N} W_e^t \tag{1}$$

Each edge node aggregates its clients:

$$W_e^t = \sum_{i \in e} \frac{N_i}{N_e} W_i^t \tag{2}$$

where W_g^t, W_e^t, W_i^t denote global, edge, and client parameters; N_e, N_i, N denote edge, client, and total dataset sizes. The global server receives only $m = 2$ edge models per round versus $n = 6$ in flat FL, reducing cloud communication by 67% at current scale. Since communication is widely recognized as the principal bottleneck in FL [20], this structural reduction is the core scalability gain of the hierarchy. Table I summarizes layer responsibilities.

TABLE I
LAYER RESPONSIBILITIES

Layer	Responsibilities
Global Server	Global aggregation, round coordination, model management
Edge Nodes	Edge aggregation, client management, update forwarding
Docker Clients	Local training, preprocessing, private data retention

C. Communication Workflow

Fig. 2 illustrates the communication workflow. The *FTL initialization phase* runs once: the Anomaly Transformer is pretrained, and weights are broadcast from global server through edge nodes to all clients. The *federated optimization phase* repeats for $T_{\max} = 20$ rounds: (1) global server broadcasts W_g^t to both edge nodes; (2) each edge forwards W_g^t to its three clients; (3) each client runs $E = 5$ local training epochs; (4) clients return updated W_i^t to their edge node; (5) edge nodes aggregate via Eq. (2) and transmit W_e^t to the global server; (6) global server computes W_g^{t+1} via Eq. (1).

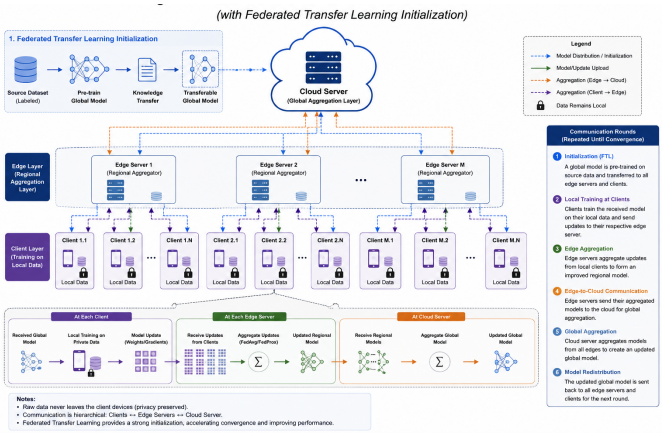


Fig. 2. Hierarchical Federated Communication Workflow: FTL initialization phase (once) followed by the six-stage per-round federated optimization phase.

IV. Methodology

A. Problem Formulation

Each client c_i holds private dataset $D_i = \{(x_j, y_j)\}_{j=1}^{N_i}$ where $x_j \in \mathbb{R}^{T \times F}$ is a time-series window and $y_j \in \{0, 1\}$ is the binary anomaly label. The two-level hierarchical objective is:

$$\min_w F(w) = \sum_{e=1}^{m} \frac{N_e}{N} \left[\sum_{i \in e} \frac{N_i}{N_e} F_i(w) \right] \tag{3}$$

where $F_i(w) = \frac{1}{N_i} \sum_j \ell(x_j, y_j; w)$ is the local binary cross-entropy loss. This two-level formulation decomposes the global objective into localized edge-level sub-problems, ensuring the hierarchical aggregation in Eqs. (2)–(1) recovers an unbiased estimate of the global optimum.

B. Mathematical Characterization of Non-IID Data and Ill-Conditioning

To precisely characterize the optimization challenges across the hierarchical edge-cloud architecture, we formalize the twin bottlenecks of statistical heterogeneity (non-IID data) and local objective ill-conditioning.

1) Statistical Heterogeneity Across Hierarchical Tiers: Let $\mathcal{P}_i$ denote the true underlying data distribution of client i, and let $\mathcal{P}_e = \sum_{i \in e} \frac{N_i}{N_e} \mathcal{P}_i$ represent the aggregate distribution visible to edge node e. The global objective seeks to minimize the expected loss over the global distribution $\mathcal{P}_g = \sum_{e=1}^{m} \frac{N_e}{N} \mathcal{P}_e$. In a realistic edge deployment, statistical divergence manifests at two distinct boundaries:

1) **Intra-Edge Divergence:** The statistical discrepancy among clients assigned to the same edge node, quantified via the Total Variation (TV) distance:

$$\delta_e = \sum_{i \in e} \frac{N_i}{N_e} \mathcal{D}_{\text{TV}}(\mathcal{P}_i, \mathcal{P}_e) \tag{4}$$

2) **Inter-Edge Divergence:** The macro-level distribution shift between separate edge domains and the global average:

$$\Delta_g = \sum_{e=1}^{m} \frac{N_e}{N} \mathcal{D}_{\text{TV}}(\mathcal{P}_e, \mathcal{P}_g) \tag{5}$$

When $\delta_e > 0$ and $\Delta_g > 0$, the local objectives $F_i(w)$ drift significantly from both the localized edge target $F_e(w)$ and the global optimum $F(w)$, skewing local gradient steps away from the true global descent trajectory.

2) Ill-Conditioned Local Objectives: In addition to statistical drift, the structural composition of non-IID partitions often leads to highly ill-conditioned local optimization surfaces. For a client i, let $H_i(w) = \nabla^2 F_i(w)$ denote the local Hessian matrix of the loss function. The local condition number κ_i is defined as the ratio of the maximum to minimum eigenvalues of the Hessian:

$$\kappa_i = \frac{\lambda_{\max}(H_i(w))}{\lambda_{\min}(H_i(w))} \tag{6}$$

Under severe non-IID conditions—such as extreme label skew or highly unbalanced feature spaces present in the IoT datasets—the local loss surfaces exhibit high anisotropy (i.e., $\kappa_i \gg 1$). Consequently, standard first-order local optimizers suffer from oscillatory behavior along directions of high curvature, slow progress along valleys of low curvature, and accelerated client drift during the local steps before edge aggregation occurs. These are the heterogeneous, ill-conditioned objectives that arise across non-IID edge-cloud clients, which our hierarchical framework and adaptive optimization strategies aim to solve.

C. Hierarchical Execution and Complexity

The three stages compose into a single training loop as detailed in Algorithm 1. The global server initializes w_g^0. In every round t it broadcasts w_g^t to the edges, each edge forwards it to its clients, and every client runs κ local steps of the selected optimizer on its own partition—mitigating the high condition number κ_i—and returns the updated model. Each edge then forms its aggregate W_e^t by (2) and transmits only that single model to the server, neutralizing the localized intra-edge divergence δ_e. The server then produces w_g^{t+1} by (1) and starts the next round.

Since the wide-area link carries one model per edge rather than one per client, the per-round cloud-bound traffic scales with the number of edges m instead of the number of clients n. For the deployment studied here, with $m = 2$ and $n = 6$, this already cuts the cloud-bound update count by a factor of three, and the gap widens exponentially as clients are added under each edge. This is the structural origin of the communication savings, and because the aggregation weights are strictly preserved, it comes at no cost to the optimization target established in (3).

D. Federated Transfer Learning Initialization

Conventional FL random initialization requires 8–12 rounds in non-IID settings before meaningful accuracy. The FTL stage pretrains the Anomaly Transformer on labeled data yielding W^{pre}, distributed through the hierarchy so every client begins round 1 with $W_i^0 = W^{pre}$. In FTL mode the backbone is *frozen*; only the classification head $\Theta_i^{train} = \{W_l, b_l\}_{l=1}^{3}$ is updated, reducing per-round communication to classifier parameters only and preventing catastrophic forgetting of pretrained temporal representations.

Algorithm 1 Hierarchical Edge-Cloud Federated Learning Loop

Require: Global rounds $T_{\max}$, local epochs E, client datasets D_i.
Require: Aggregation weights N_i, N_e, N.
Server Initialize: Broadcast W_g^0 to all edge nodes $e \in \{1, \ldots, m\}$.
for each global round $t = 0, 1, \ldots, T_{\max} - 1$ **do**
  for each edge node $e = 1, \ldots, m$ **in parallel do**
    Receive W_g^t from the global cloud server.
    Distribute W_g^t to all connected clients $i \in e$.
    for each client $i \in e$ **in parallel do**
      Run κ local steps (i.e., E epochs) via selected optimizer (FedAvg, FedProx, or FedAdaProx).
      Return updated local model W_i^t to edge node e.
    end for
    Form edge-level aggregate model via (2):
    $W_e^t \leftarrow \sum_{i \in e} \frac{N_i}{N_e} W_i^t$
    Transmit W_e^t across the WAN to the cloud server.
  end for
  Cloud Server Aggregation:
  Compute next global state W_g^{t+1} via (1):
  $W_g^{t+1} \leftarrow \sum_{e=1}^{m} \frac{N_e}{N} W_e^t$
end for

E. Anomaly Transformer

The Anomaly Transformer [12] detects anomalies via association discrepancy: normal points maintain Gaussian-concentrated temporal associations while anomalous points exhibit disrupted patterns. Given $X \in \mathbb{R}^{T \times F}$:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{7}$$

where $Q = XW^Q$, $K = XW^K$, $V = XW^V$. Series association S is the row-normalized attention matrix. Learnable Gaussian prior:

$$P(i, j) = \frac{1}{\sqrt{2\pi}\sigma_i} \exp\left(-\frac{|i - j|^2}{2\sigma_i^2}\right) \tag{8}$$

Symmetric association discrepancy and anomaly score:

$$D(P, S) = \frac{1}{T} \sum_{i=1}^{T} [\text{KL}(P_{i,\cdot} \| S_{i,\cdot}) + \text{KL}(S_{i,\cdot} \| P_{i,\cdot})] \tag{9}$$

$$A(x_t) = \|P_t - S_t\|, \quad \mathcal{L} = \mathcal{L}_{rec} + \lambda \mathcal{L}_{assoc} \tag{10}$$

For federated binary classification, the encoder output $Z \in \mathbb{R}^{T\times d}$ is pooled and classified:

$$h = \text{MeanPool}(Z) + \text{MaxPool}(Z) \tag{11}$$

$$\hat{y} = W_3 \cdot \text{ReLU}(W_2 \cdot \text{ReLU}(W_1 \cdot h + b_1) + b_2) + b_3 \tag{12}$$

$$\mathcal{L}_{BCE} = -\frac{1}{N}\sum_j [y_j \log \sigma(\hat{y}_j) + (1-y_j)\log(1-\sigma(\hat{y}_j))] \tag{13}$$

Table II presents the model configuration.

TABLE II
ANOMALY TRANSFORMER MODEL CONFIGURATION

Hyperparameter	Value
Window Size (T)	128
Input Features (F)	115 (N-BaIoT) / 9 (UCI-HAR)
Model Dimension (d_{model})	128
Attention Heads	4
Encoder Layers	2
Feedforward Dimension	256
Dropout	0.1
Classifier Dimensions	d→64→32→1

F. Federated Optimization

FedAvg [1] updates the global model by weighted averaging: $W_g^t = \sum_i \frac{N_i}{N} w_i^t$. Communication-efficient but susceptible to client drift [19] under non-IID distributions, a manifestation of the heterogeneous ill-conditioned objectives across edge-cloud clients.

FedProx [13] constrains local updates via proximal regularization:

$$\tilde{F}_i(w) = F_i(w) + \frac{\mu}{2}\|w - w_g\|^2 \tag{14}$$

The proximal term anchors local optimization to the global model, reducing weight divergence [21] caused by heterogeneous local objectives. However, a fixed μ cannot accommodate varying client divergence profiles.

FedAdaProx follows the adaptive federated optimization principle of Reddi et al. [18], applying Adam-style first- and second-moment estimates during local updates:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \tag{15}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{16}$$

$$w^{t+1} = w^t - \frac{\eta_t \cdot m_t}{\sqrt{v_t} + \epsilon} \tag{17}$$

The key innovation is the adaptive per-client proximal coefficient addressing the heterogeneous, ill-conditioned objectives across non-IID edge-cloud clients:

$$\mu_k = \mu_0 + \alpha \sum_l \|w_k^l - w_g^l\| + \beta \cdot \Delta\mathcal{L}_k \tag{18}$$

where μ_0 is the base coefficient, α weights parameter divergence from the global model over layers l, and β weights loss instability $\Delta\mathcal{L}_k$ from the previous round. Clients with high divergence or unstable loss receive stronger regularization; well-converged clients retain more optimization freedom. Edge and global aggregation follow Eqs. (2) and (1). Algorithm 2 details the procedure.

Algorithm 2 FedAdaProx Local Client Training

Require: w_g, D_k, μ_0, η, E, α, β
$w_k \leftarrow w_g$; compute $\delta = \sum_l \|w_k^l - w_g^l\|$
$\mu_k = \mu_0 + \alpha \cdot \delta + \beta \cdot \Delta\mathcal{L}_k$
for epoch $e = 1$ to E **do**
  for each mini-batch $(x, y) \in D_k$ **do**
    $\mathcal{L} = \mathcal{L}_{BCE}(w_k; x, y) + \frac{\mu_k}{2}\|w_k - w_g\|^2$
    Update w_k via adaptive step (Eqs. 10–12)
  end for
end for
$\Delta\mathcal{L}_k \leftarrow \mathcal{L}$;
return w_k, $\Delta\mathcal{L}_k$

V. IMPLEMENTATION

Technology Stack. Python 3.10, PyTorch (model training), Flask (REST APIs over HTTPS), Docker (client containers), Microsoft Azure (VM infrastructure), Ubuntu 22.04 LTS, NumPy/Pandas, Scikit-learn.

Azure Deployment. Global server: 4 vCPUs, 16 GB RAM. Two edge nodes: 4 vCPUs, 8 GB RAM each. Six Docker clients distributed three per edge node, communicating via JSON-serialized HTTPS APIs.

We utilize 2 datasets:

- **UCI-HAR** [14]: Triaxial accelerometer and gyroscope data (50 Hz) from 30 subjects across 6 activities, preprocessed into 128-sample windows yielding 9 features. Subject-based non-IID partitioning assigns different subject subsets to each client.
- **N-BaIoT** [15]: Network traffic statistics (115 features) from 9 real IoT devices under normal and Mirai/BASHLITE botnet attack conditions—the dataset used in the pioneering DIoT federated IDS study [31]. Cross-device split: trained on devices 1–7, evaluated on held-out devices 8–9. Non-IID partitioning distributes different device profiles and attack/normal ratios across the six clients.

Model & Training. Window $T = 128$, $d_{model} = 128$, 4 heads, 2 encoder layers, feedforward 256, dropout 0.1, classifier d→64→32→1. Adam, lr= 0.001, batch 32, $E = 5$ local epochs, 20 rounds, FedProx $\mu = 0.01$, FedAdaProx $\mu_0 = 0.01$, $\alpha = 0.1$, $\beta = 0.05$.

VI. EXPERIMENTAL RESULTS

A. UCI-HAR Results

Figs. 3–6 show per-round metrics on UCI-HAR. All algorithms converge within 3–5 rounds due to FTL initialization. Table III reports averages. FedProx achieves the highest accuracy (95.61%) and F_1 (95.41%), benefiting from proximal regularization controlling client drift under subject-based heterogeneity. FedAvg (95.11%, 94.83% F_1) is competitive due to FTL reducing effective drift. FedAdaProx (92.91%, 92.28% F_1) shows minor instability on the smaller 9-feature dataset where divergence signals are noisier.

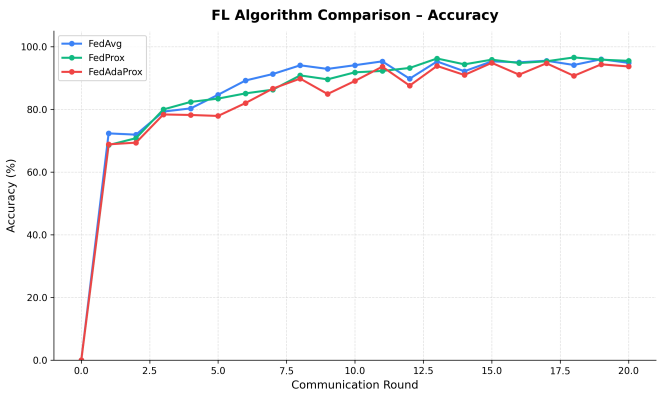


Fig. 3. Accuracy on UCI-HAR over 20 communication rounds.

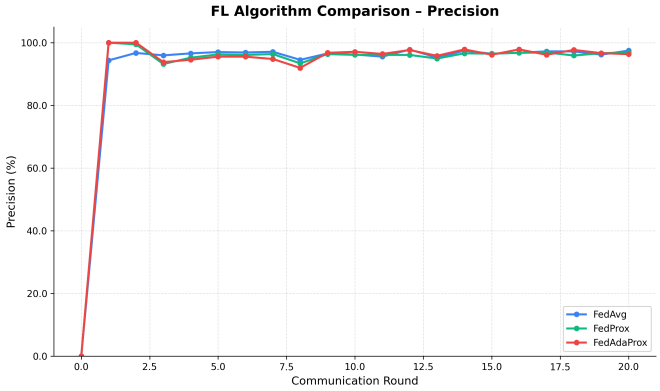


Fig. 4. Precision on UCI-HAR over 20 communication rounds.

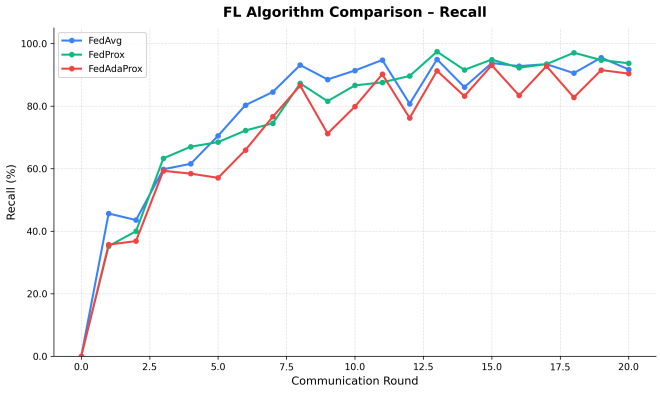


Fig. 5. Recall on UCI-HAR over 20 communication rounds.

TABLE III
AVERAGE PERFORMANCE ON UCI-HAR

Algorithm	Acc (%)	Prec (%)	Rec (%)	F_1 (%)
FedAvg	95.11	96.98	92.80	94.83
FedProx	**95.61**	96.62	**94.24**	**95.41**
FedAdaProx	92.91	**96.94**	88.18	92.28

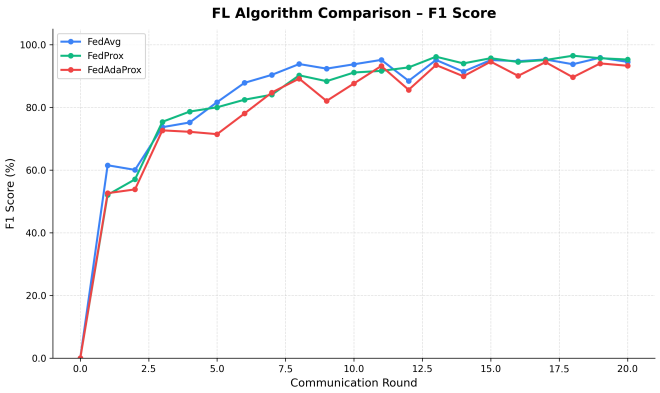


Fig. 6. F_1-score on UCI-HAR over 20 communication rounds.

B. N-BaIoT Results

Figs. 7–10 show N-BaIoT results. All algorithms converge within 2–3 rounds. Table IV reports averages. FedAdaProx achieves the best performance (98.58%, 98.52% F_1), followed by FedProx (98.38%, 98.32%) and FedAvg (97.97%, 97.90%). All algorithms achieve 99.00% recall, demonstrating near-perfect botnet detection sensitivity. FedAdaProx's adaptive μ_k (Eq. 18) effectively handles the strong device-specific heterogeneity of the 115-feature non-IID partitions, providing stronger regularization to high-divergence clients and more freedom to well-aligned clients.

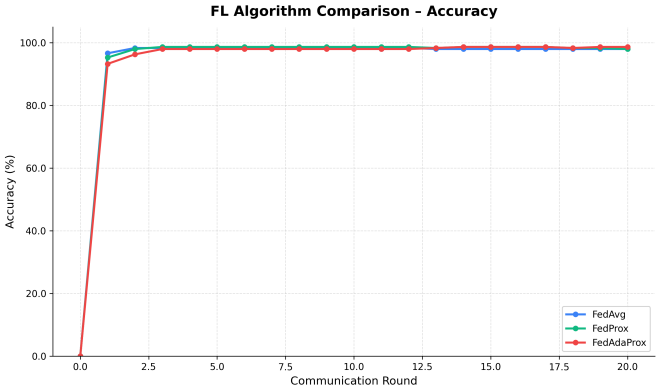


Fig. 7. Accuracy on N-BaIoT over 20 communication rounds.

TABLE IV
AVERAGE PERFORMANCE ON N-BAIOT

Algorithm	Acc (%)	Prec (%)	Rec (%)	F_1 (%)
FedAvg	97.97	95.89	**99.00**	97.90
FedProx	98.38	96.69	**99.00**	98.32
FedAdaProx	**98.58**	**97.09**	**99.00**	**98.52**

C. Comparative and Scalability Analysis

Table V consolidates cross-dataset results. FedProx excels on UCI-HAR where fixed proximal regularization suffices

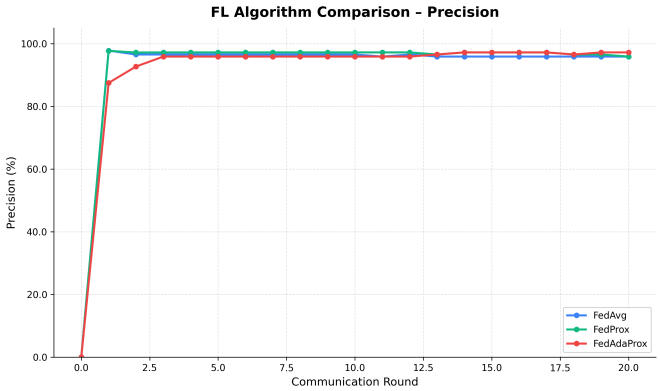


Fig. 8. Precision on N-BaIoT over 20 communication rounds.

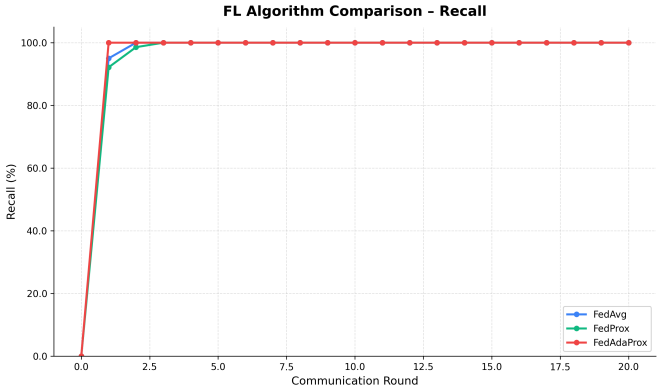


Fig. 9. Recall on N-BaIoT over 20 communication rounds.

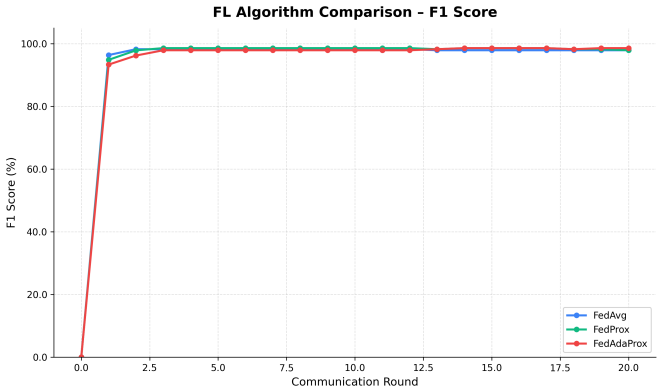


Fig. 10. F_1-score on N-BaIoT over 20 communication rounds.

for moderate sensor heterogeneity. FedAdaProx excels on N-BaIoT where adaptive per-client μ_k handles strong device-specific distributional differences. FedAvg remains competitive on both, aided by FTL pre-reducing drift.

TABLE V
OVERALL COMPARATIVE PERFORMANCE

Dataset	Alg.	Acc	Prec	Rec	F_1
UCI-HAR	FedAvg	95.11	96.98	92.80	94.83
UCI-HAR	FedProx	**95.61**	96.62	**94.24**	**95.41**
UCI-HAR	FedAdaProx	92.91	96.94	88.18	92.28
N-BaIoT	FedAvg	97.97	95.89	99.00	97.90
N-BaIoT	FedProx	98.38	96.69	99.00	98.32
N-BaIoT	FedAdaProx	**98.58**	**97.09**	99.00	**98.52**

The hierarchical architecture reduces cloud communication from $O(n)$ to $O(m)$, cutting cloud-bound updates by $3\times$ at current scale—growing further as clients are added under each edge—at no cost to the optimization target in Eq. (3).

D. Discussion

FTL Impact. All algorithms exceed 80% accuracy within 2–3 rounds versus 8–12 rounds typical of random initialization in non-IID FL. FTL's pretrained backbone provides universal temporal representations transferable across sensor and network traffic domains, confirming domain-agnostic transferability.

Algorithm Selection. FedProx suits moderate heterogeneity with lower-dimensional data. FedAdaProx suits high-dimensional settings with strongly varying client divergence. FedAvg is a strong baseline when FTL pre-reduces drift.

Anomaly Transformer. The 99.00% recall on N-BaIoT confirms strong botnet detection through association discrepancy learning, capturing long-range temporal disruptions that LSTM encoder-decoders [29] and reconstruction autoencoders [9] cannot model over full 128-step windows.

Privacy and Deployment. Sharing parameters rather than raw data reduces privacy exposure, though formal guarantees require secure aggregation [32] or DP-SGD [33]—planned extensions. The Azure + Docker deployment distinguishes this work from simulation-only HFL studies.

VII. CONCLUSION

This paper presented a hierarchical edge-cloud FL framework integrating FTL initialization, the Anomaly Transformer, and adaptive federated optimization for distributed time-series anomaly detection, deployed on Microsoft Azure. The three-tier hierarchy reduces cloud communication by $O(n/m)$ over flat FL. FedProx achieves 95.61% F_1 on UCI-HAR; FedAdaProx achieves 98.52% F_1 on N-BaIoT, both under non-IID conditions. FTL consistently reaches above 80% accuracy within 2–3 rounds.

Future work will investigate: (1) secure aggregation [32] and DP-SGD [33]; (2) efficient attention backbones [26]; (3) update compression [20] for WAN links; (4) staleness-tolerant aggregation [23]; and (5) robustness against Byzantine attacks.

REFERENCES

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.

[2] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1–2, pp. 1–210, 2021.

[3] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.

[4] Q. Li, Z. Wen, and Z. Wu, "A survey on federated learning systems," *IEEE Trans. Knowl. Data Eng.*, 2021.

[5] D. Nguyen, M. Ding, and P. Pathirana, "Federated learning for IoT: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1622–1658, 2021.

[6] L. Liu, J. Zhang, S. H. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *Proc. IEEE ICC*, 2020, pp. 1–6.

[7] W. Lim, N. Luong, and D. T. Hoang, "Federated learning in mobile edge networks: A survey," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 3, pp. 2031–2063, 2020.

[8] G. Pang, C. Shen, and L. Cao, "Deep learning for anomaly detection: A review," *ACM Comput. Surveys*, vol. 54, no. 2, 2021.

[9] C. Zhou and R. Paffenroth, "Anomaly detection with robust deep autoencoders," in *Proc. ACM SIGKDD*, 2022.

[10] A. Vaswani et al., "Attention is all you need," in *Adv. NeurIPS*, 2017.

[11] Q. Wen, T. Zhou, and C. Zhang, "Transformers in time series: A survey," in *Proc. IJCAI*, 2023.

[12] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *Proc. ICLR*, 2022.

[13] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proc. MLSys*, vol. 2, pp. 429–450, 2020.

[14] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *Proc. ESANN*, 2013.

[15] Y. Meidan et al., "N-BaIoT: Network-based detection of IoT botnet attacks," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, 2018.

[16] D. Merkel, "Docker: Lightweight linux containers," *Linux J.*, 2014.

[17] G. Yan, S. Yang, and W. Du, "FedRACE: A hierarchical framework for robust federated learning," in *Adv. NeurIPS*, 2025.

[18] S. J. Reddi et al., "Adaptive federated optimization," in *Proc. ICLR*, 2021.

[19] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. ICML*, 2020, pp. 5132–5143.

[20] J. Konecny, H. B. McMahan, F. X. Yu, P. Richtarik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *arXiv:1610.05492*, 2016.

[21] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," *arXiv:1806.00582*, 2018.

[22] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

[23] Q. Wu et al., "HiFlash: Communication-efficient hierarchical federated learning with adaptive staleness control," *IEEE Trans. Parallel Distrib. Syst.*, 2023.

[24] M. Sarhan, W. W. Lo, S. Layeghy, and M. Portmann, "HBFL: A hierarchical blockchain-based federated learning framework for IoT intrusion detection," *Comput. Electr. Eng.*, vol. 103, 2022.

[25] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series," *Proc. VLDB Endow.*, vol. 15, no. 6, pp. 1201–1214, 2022.

[26] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI*, vol. 35, 2021, pp. 11106–11115.

[27] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Adv. NeurIPS*, 2021.

[28] Z. Z. Darban, G. I. Webb, S. Pan, C. C. Aggarwal, and M. Salehi, "Deep learning for time series anomaly detection: A survey," *ACM Comput. Surv.*, 2024.

[29] P. Malhotra et al., "LSTM-based encoder-decoder for multi-sensor anomaly detection," in *Proc. ICML Anomaly Detection Workshop*, 2016.

[30] A. Deng and B. Hooi, "Graph neural network-based anomaly detection in multivariate time series," in *Proc. AAAI*, vol. 35, 2021, pp. 4027–4035.

[31] T. D. Nguyen et al., "DIoT: A federated self-learning anomaly detection system for IoT," in *Proc. IEEE ICDCS*, 2019, pp. 756–767.

[32] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM CCS*, 2017, pp. 1175–1191.

[33] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM CCS*, 2016, pp. 308–318.